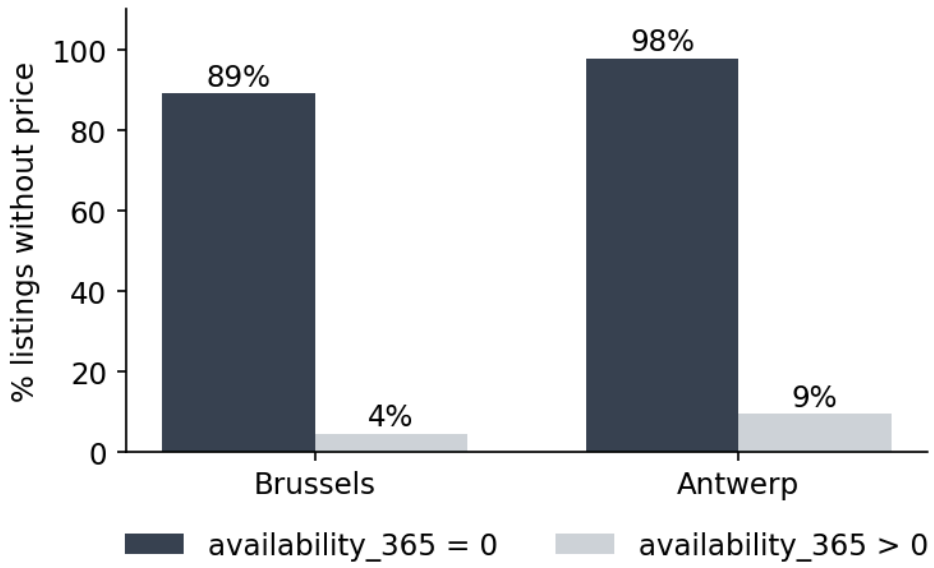
GROUP ASSIGNMENT 2 · BIG DATA · REGION 12: BENELUX (EUR) · TEAM BOBERRMDT

Our pipeline ran a new city without a single error, and that was the problem

Lakehouse and price model built on Brussels (A) and Antwerp (B), then run unchanged on the new client Amsterdam (C)

Matvii Shumylovych · Dmytro Hrebenyuk

Price is missing for a reason, outliers are mostly placeholders, and four columns are the price



Missing, explained: no price = no bookable night, so no quote. That is not random, so we **don't impute**: the rows stay in silver for supply metrics and are left out of training.

Outliers, explained: above €1,000 (0.5 %) are mostly 0-1-review placeholders (a 1-guest room at €1,500+); they are **flagged, not deleted**. A 25-bedroom, 2-guest flat is a typo: value set to NULL, row kept.

Leakage review: found by exact equality, not by correlation

column	relation to price	verdict
price_quote_price_per_night	= price (99.96 %)	leak
price_quote_total_price	= price × nights	leak
price_quote_raw	JSON that contains the price	leak
estimated_revenue_l365d	= price × occupancy	leak
availability_*, occupancy	a consequence of the price	excluded
neighbourhood	target-encoded inside CV folds only	used

Reproducible: the notebook `00_eda_brussels_antwerp` rebuilds every number from the raw files and never loads Amsterdam.

Three layers, each with one job, and one pipeline where the city is the only parameter

Inside Airbnb
city → latest snapshot URL, found on the index page

→

Bronze
keep it as it came
raw file in a UC volume, all strings, lineage

→

Silver
make it trustworthy
column contract, try_cast, price parsing, dedup, outlier flags, DQ checks

→

Gold
answer questions
features, city / neighbourhood / room-type metrics, predictions, model metrics

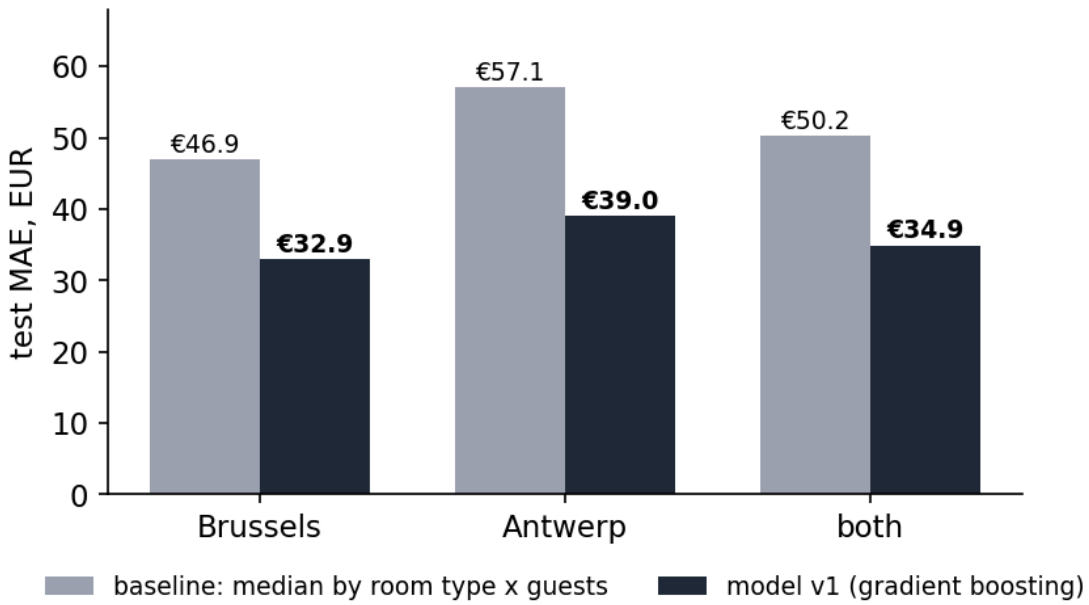
→

Dashboard
Genie

- **Job** `airbnb_city_etl(city)` : bronze → silver → gold for one city
- **Job** `airbnb_lakehouse(cities)` : for-each city ETL → train (optional) → for-each score → report
- Each city overwrites only its own partition (`replaceWhere`): runs are idempotent and cities run in parallel

Proof that it really runs per city: Amsterdam was onboarded with one parameter value,
`cities=amsterdam, retrain=false, model_version=v1`
No code change: `git diff week1-final` was empty before the run.

On held-out listings the model beats the baseline by 30 % in both cities



- **Honest test set**: 20 % of listings, picked by a hash of the id, set aside *before* any choice; the model was chosen by 5-fold CV on the other 80 %
- **Baseline**: median training price for the same room type and number of guests
- **Candidates (CV MAE)**: ridge €42.9, random forest €38.6, **gradient boosting €37.9**; target is log(price)
- **Features we can justify**: everything a host knows *before* setting a price: guests, bedrooms, room type, minimum nights, amenities, reviews, distance to centre, neighbourhood
- **No leakage**: the four leak columns and availability are excluded, the neighbourhood encoder is fitted inside CV
- **Weak spot**: above €400 the error is €173 and prices are under-predicted

Amsterdam, first run: exit code 0, no warnings, and 95 % of homes under-priced

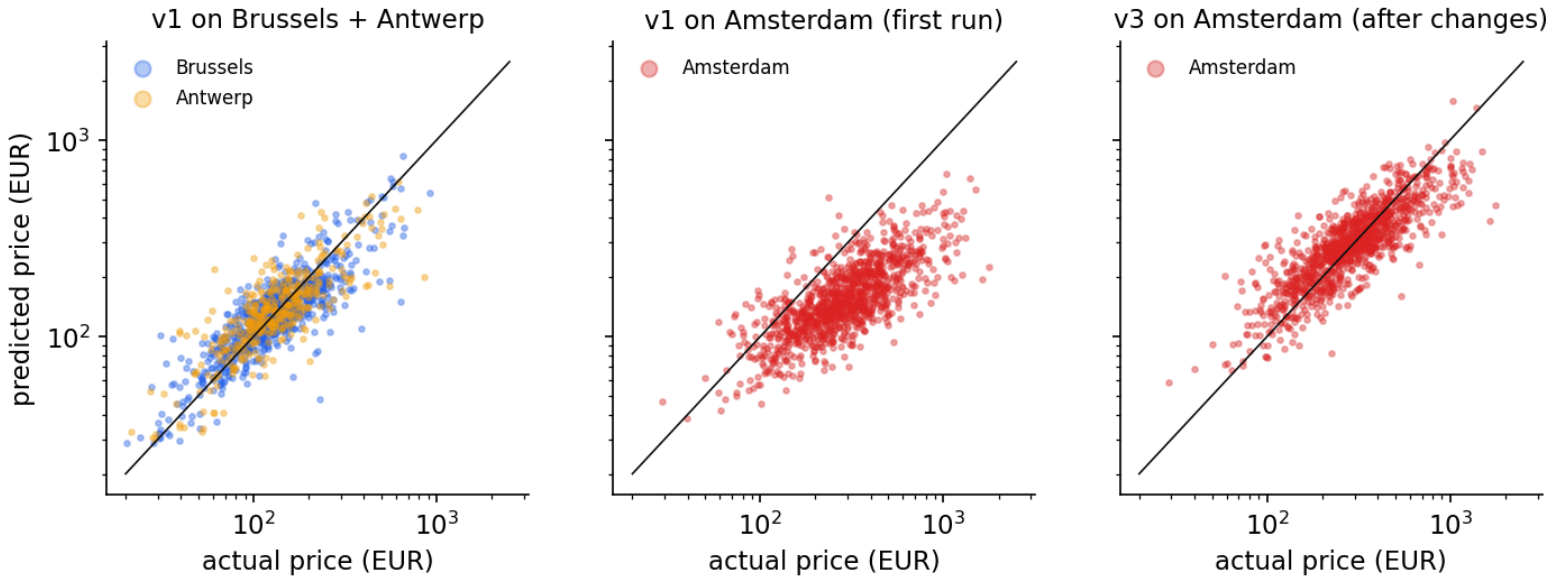
0
errors or warnings

€158.6
MAE on Amsterdam
(€34.9 on Brussels + Antwerp)

−€155
mean error: 95 % under-priced;
no prediction above €601

43 %
missing prices passed silently
(16-20 % in A/B)

100 %
neighbourhoods the model
has never seen



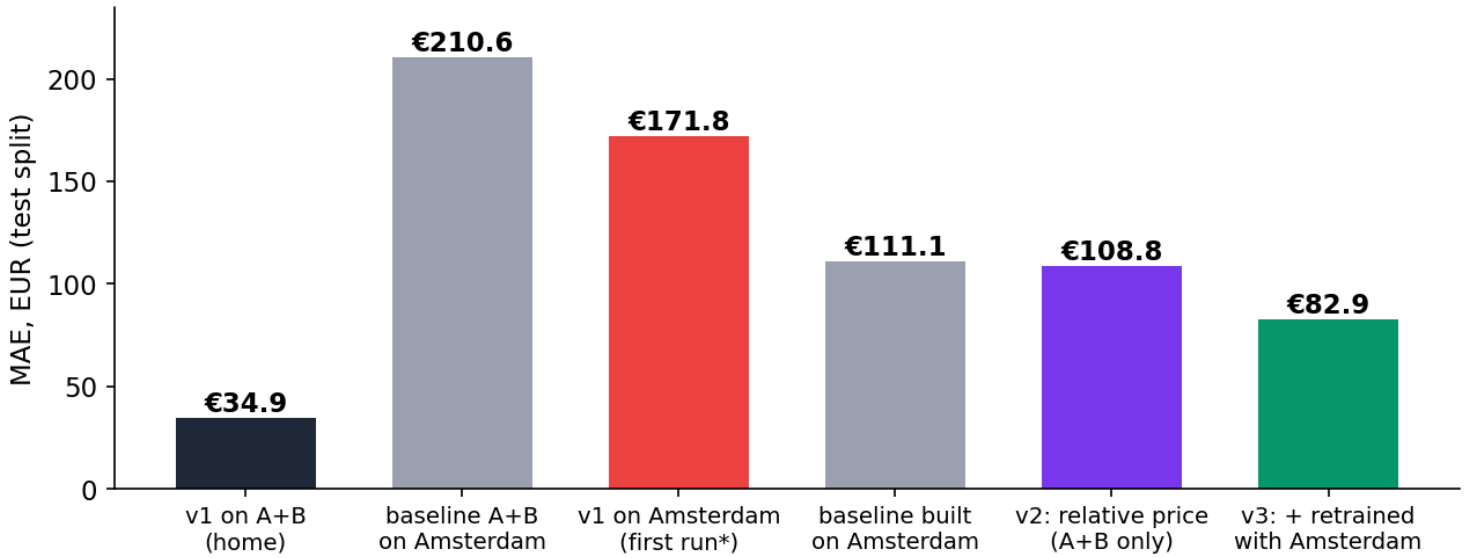
Gradient-boosted trees can't predict above the prices they were trained on, so v1 flattens at Belgian levels (middle panel). The pipeline coped with changes in *format* but didn't notice changes in *distribution*.

Every change since the freeze, sorted into pipeline fixes and model changes

#	kind	change	why
P1	pipeline	outlier ceiling = 8× the city's median (was a fixed €1,000)	the rule encoded Belgian prices; it deleted 103 genuine Amsterdam homes
P2	pipeline	data-quality expectations → `dq_metrics` pass / warn	a 43 % missing target went through without a word
P3	pipeline	scoring coverage: unseen city, unseen neighbourhoods, price level	the model has to say when it is extrapolating
P4-P6	pipeline	DQ per layer; `a+b+c` city lists; job dependency fix	found while deploying, not caused by Amsterdam
M1	model	relative target: log(price / city price index)	learn structure, not Belgian euros
M2	model	drop the one-hot `city`	it means nothing for an unseen city
M3	model	retrain with Amsterdam's 80 % training split (v3)	its structure differs; its 20 % test split stays untouched

Left unchanged because it held up on unseen data: URL resolution, download, column contract, try_cast, parsing, dedup, the split. Every change is tagged in the code and listed by `git diff week1-final`.

Most of the gap is price level; what's left is the same 25 % error on prices twice as high



≈ €60: price level
Rescaling v1 by Amsterdam's median gives €112.7. v2 gets €108.8, but a baseline built on Amsterdam alone scores €111.1

≈ €26: price structure
Bigger entire-home premium, steeper drop with distance. v3 learns it: €82.9, 25 % below that baseline

The rest: proportional
Relative error is 26 % for v3 on Amsterdam vs 23 % for v1 at home. Same relative error, on prices 2.2× higher

* €171.8 = v1 after P1 puts the 103 expensive homes back. The original first run (€158.6) looked better only because those homes had been dropped.

Live demo: each dashboard page opens with its answer, and Genie returns the numbers we expect

AI/BI dashboard: page titles state the finding

1. **City overview**: “Amsterdam costs 2.3× Belgium (median €285 vs €125-130), and 42 % of its calendars are closed”
2. **Cross-city comparison**: “Brussels and Antwerp price alike; what differs is the mix (Antwerp: 84 % entire homes, 61 % multi-listing hosts)”
3. **Model**: “v1 beats the baseline by 30 % at home but under-priced Amsterdam by €168; the relative model retrained on Amsterdam (v3) cuts its MAE from €172 to €83”, plus the pipeline warnings table

Genie: question → expected answer

we ask	it should answer
Median price of entire homes in City A?	€138 (Brussels, 3,509 homes)
Neighbourhoods in City B with the most listings?	Historisch Centrum 397, Amandus-Atheneum 219, Theaterbuurt-Meir 194
How did the model do on the new city?	v1 €172 → v2 €109 → v3 €83

How we got it right: 5 pre-aggregated gold tables, “City A/B/C” mapped in the instructions, a trusted SQL example for each question, UC comments on every column, and test-split-only evaluation built into the instructions.

A crash is easy to spot. We fixed the run that succeeded and was still wrong

- The week-1 pipeline handled **format** changes and was blind to **distribution** changes
- Now it **warns** when a city doesn't look like the training cities (P2, P3)
- The model learns **price relative to the city**, which transfers (M1, M2)
- Production routing: v1 for Brussels and Antwerp, v3 for Amsterdam

member	contribution
Matvii Shumylovych	EDA and leak review, medallion pipeline, Databricks bundle and jobs, Amsterdam first run, pipeline fixes P1-P6, Genie
Dmytro Hrebenyuk	Price model v1 and model cards, model changes M1-M3 (v2, v3), city C report, AI/BI dashboard, presentation

Everything is in the repo: `docs/city_c/report.md`, model cards, logs of each run.